Clock Synchronization and Clock Skew

why timestamps can't order events across machines

Watch the 4-min video

In one line: every machine's clock drifts, and NTP only keeps them within milliseconds of each other, so timestamps from different servers can't reliably order events; use monotonic clocks for durations, and logical clocks, version numbers or bounded-uncertainty clocks when order matters.

1 Later isn't always later

Two servers write to the same record a few milliseconds apart, and you keep the one with the later timestamp. Simple, right? But if the second server's clock is 50 ms behind, the newer write carries the older timestamp and is silently thrown away. Across machines, **timestamps lie** by small, unpredictable amounts.

2 Why clocks disagree

Each server counts time with a quartz crystal whose frequency depends on temperature, so clocks **drift** by roughly tens of microseconds per second. The difference between two clocks at a moment is called **clock skew**. **NTP** (Network Time Protocol) periodically corrects each clock against time servers, but network delays limit its accuracy to about a millisecond on a good LAN, and tens of milliseconds over the internet. Corrections can also make a clock **jump backwards**, and leap seconds add their own surprises.

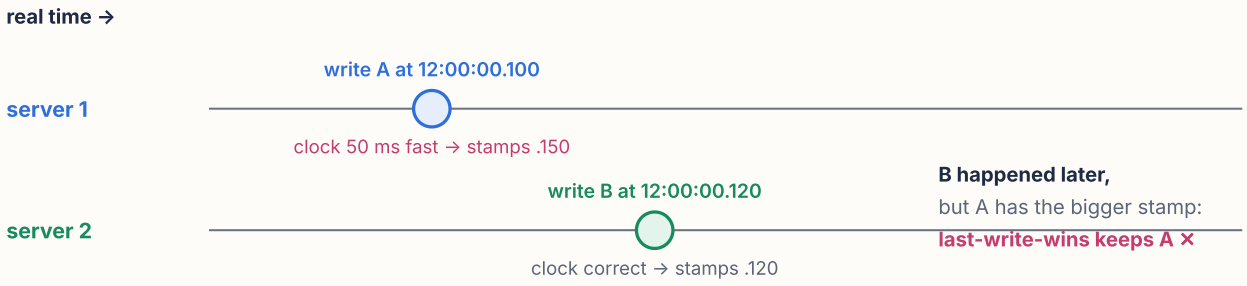


Figure 1. A 50 ms skew is enough to reverse the order of two writes when you compare timestamps.

WORKED EXAMPLE: LAST-WRITE-WINS LOSES AN UPDATE

A key-value store resolves conflicts with **last-write-wins** (LWW): the write with the highest timestamp wins. A user updates their address on server 1, whose clock runs 50 ms fast, then 20 ms later fixes a typo through server 2, whose clock is correct. The typo fix carries the smaller timestamp, so every replica keeps the version with the typo, and nothing reports an error. Fixes: use a version number or compare-and-set on the record, route a user's writes to one leader, or use logical or hybrid clocks (lessons 6 and 7).

3 Two kinds of clock

Operating systems offer a **wall clock** (time of day, e.g. `time.time()`) and a **monotonic clock** (e.g. `time.monotonic()`). The monotonic clock never goes backwards, which makes it the right choice for

timeouts, rate limits and latency measurement. It means nothing across machines, though.

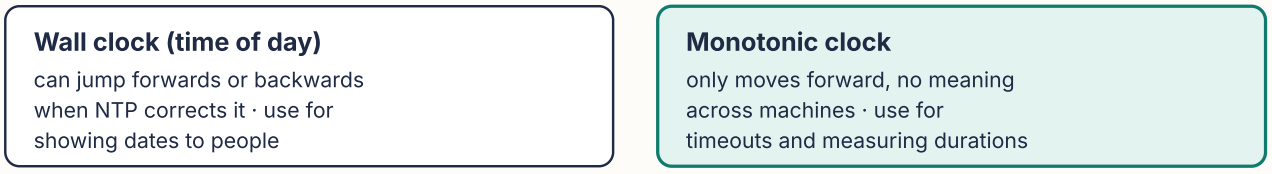


Figure 2. Use the monotonic clock to measure; use the wall clock only to display or log a date.

4 When you do need real time: bounded uncertainty

Google Spanner's **TrueTime** uses GPS receivers and atomic clocks in every data center and returns an interval, not a point: the true time is somewhere between *earliest* and *latest*, typically a few milliseconds apart. Before making a commit visible, Spanner **waits out** the uncertainty, so commit timestamps respect real-time order. AWS Time Sync and PTP give similar tight bounds. Without hardware like this, assume skew of at least several milliseconds.



Figure 3. TrueTime admits its uncertainty and waits it out, which makes timestamp order safe.

5 Practical rules

- Never use wall-clock timestamps from different machines to decide which write wins, unless losing a write is acceptable.
- Generate order at one place: a leader, a database sequence, or a log offset.
- Store timestamps in UTC, and monitor NTP offset on every host; alert if it exceeds a few milliseconds.
- Leases and lock expiry must allow for skew and pauses (lesson 13).

COMMON MISTAKES

- Measuring a timeout with `time.time()`, which can jump.
- Sorting events from many services by their local timestamps and trusting the order.
- Assuming NTP means 'synchronised exactly'.
- Last-write-wins on data where a lost update matters.

6 Key terms

Clock drift: a clock running slightly fast or slow

Clock skew: difference between two clocks at the same moment

NTP: protocol that corrects clocks against time servers

Monotonic clock: never goes backwards; for durations

Last-write-wins: keep the write with the highest timestamp

TrueTime: Spanner's clock API with an explicit uncertainty bound

7 Interview questions

Q1. Why can't you order events across servers by timestamp? BASIC
Clocks drift and are only roughly synchronised by NTP, so skew of milliseconds or more is normal; an event that happened later can carry an earlier timestamp.

Q2. Monotonic or wall clock for a timeout? BASIC
Monotonic: it never jumps backwards when NTP corrects the time, so durations are measured correctly.

Q3. Users report that some profile updates disappear. The store uses last-write-wins across regions. Explain. SCENARIO
Clock skew between regions gives a newer write an older timestamp, so it loses the conflict. Use versioning with compare-and-set, a single leader per record, or merge-friendly data types.

Q4. How would you order events from many microservices for an audit log? SCENARIO
Don't rely on their clocks: write events to one partitioned log (a Kafka partition per entity) and use the log offset as the order, or use logical or hybrid timestamps.

Q5. How does Spanner get externally consistent timestamps? TRICKY
TrueTime returns an interval with bounded uncertainty from GPS and atomic clocks; Spanner waits until the commit timestamp is certainly in the past before making the commit visible.

Q6. Can a clock go backwards on a server? What breaks? TRICKY
Yes: NTP can step it back, or a VM can be paused and resumed. Durations measured with the wall clock go negative, TTLs and leases misbehave, and IDs based on time can collide.

Next: SD-H-006 · Logical Clocks (Lamport) backendroadmap.com